

PROJECT REPORT

STUDENT PERFORMANCE ANALYZER

Submitted By

Name: Manas Pandey

Registration ID: 26BCE10001

Branch: CSE – Educational Technology

1.

The Student Performance Analyzer is a Python-based command-line application developed to manage and analyze student performance.

In institutions student performance involves multiple factors such as marks, attendance, grades and subject-wise performance. Managing this information manually can be time-consuming. May make analysis difficult.

This project provides a computerized solution for storing student information and performing basic academic analysis.

The application allows users to register students enter marks and attendance generate individual student reports and analyze class performance.

2. Problem Statement

Managing student academic records manually can make it difficult to:

- * Maintain student information
- * Calculate percentages
- * Calculate grades
- * Track attendance
- * Identify weak subjects
- * Compare student performance
- * Calculate class averages

Therefore a simple application is required to automate these basic operations and provide organized performance information.

3. Objectives

The main objectives of this project are:

1. To develop a student performance management system.

2. To store student information using Python dictionaries.
3. To record wise marks.
4. To calculate student averages and grades.
5. To calculate attendance percentages.
6. To identify the weakest subjects.
7. To classify student performance.
8. To calculate class performance statistics.
9. To practice Python programming concepts.
10. To develop a command-line application that can be executed easily.

4. Scope of the Project

The project focuses on academic performance analysis.

The current application supports:

- * Student registration

- * Marks management
- * Attendance management
- * Individual student reports
- * Student listing
- * Class performance analysis
- * Grade calculation
- * Performance classification

The current version is designed as a command-line application. Does not use a database or graphical interface.

5. Technologies Used

Technology	Purpose	
-----	-----	
Python 3	Application development	
VS Code	Code development and testing	

| Command Line Interface | User interaction |

| GitHub | Project repository and submission |

No external Python libraries are required.

6. System Requirements

Hardware Requirements

- * Computer or laptop
- * Keyboard
- * Minimum 4 GB RAM recommended
- * storage space

Software Requirements

- * Windows, Linux or macOS
- * Python 3
- * VS. Another Python-compatible editor
- * Command-line/terminal access

7. Functional Requirements

7.1 Add Student

The system accepts:

- * Roll number

- * Name

- * Branch

- * Semester

It checks whether the roll number already exists before adding the student.

7.2 Add Marks

The system allows users to enter subjects and their marks.

The application validates marks using the condition:

```text

$0 \leq \text{Marks} \leq 100$

...

Invalid values are rejected.

---

### ## 7.3 Add Attendance

The system accepts:

- \* Total number of classes

- \* Number of classes attended

Attendance is calculated as:

```text

$\text{Attendance \%} = (\text{Attended Classes} / \text{Total Classes}) \times 100$

...

The application also ensures that attended classes cannot be greater than classes.

7.4 Student Report

The system generates a report containing:

- * Name

- * Roll number

- * Branch

- * Semester

- * Subject marks

- * Total marks

- * Average

- * Grade

- * Attendance percentage

- * Attendance status

- * Best subject

- * subject

- * Performance classification

7.5 Display Students

The application displays all registered students along with their roll number, name and branch.

7.6 Performance Analysis

The system processes students with marks and calculates:

- * Highest percentage

- * Lowest percentage

- * Class average

8. Program Logic

The application begins by creating a dictionary:

```
```python
```

```
students = {}
```

...

Each students roll number acts as the key.

The students information is stored as another dictionary containing information, subjects and attendance data.

The general structure is:

```
```text
```

students

└─ Roll Number

|

└─ Name

└─ Branch

└─ Semester

└─ Subjects

└─ Total Classes

└─ Attended Classes

...

9. Major Functions

`grade(p)`

This function receives a percentage. Returns the corresponding grade.

``text

90+ → A+

80+ → A

70+ → B+

60+ → B

50+ → C

40+ → D

Below 40 → F

...

```
## `attendance(total attended)`
```

This function calculates the attendance percentage.

It also handles the case where the total number of classes is zero.

```
## `add_student()`
```

This function creates a student record.

It first checks whether the entered roll number already exists.

```
## `add_marks()`
```

This function accepts subject names and marks.

It uses input validation to ensure that marks are within the range of 0 to 100.

```
## `add_attendance()`
```

This function records attendance. Calculates the attendance percentage.

It also validates the entered attendance information.

`student\_report()`

This function generates a performance report for a selected student.

It calculates:

- * Total marks

- * Average

- * Grade

- * Attendance

- * Best subject

- * Subject

- * Performance level

`display\_students()`

This function displays all students.

```
## `performance_analysis()`
```

This function creates a list of student performance data and calculates the highest, lowest and average percentages.

```
## `)`
```

The `main()` function controls the application menu.

It repeatedly displays the operations until the user selects the Exit option.

```
# 10. Grading System
```

```
| Percentage | Grade |
```

```
| -----: | :---- |
```

```
| 90–100 | A+ |
```

```
80–89.99 | A |
```

```
| 70–79.99 | B+ |
```

60–69.99 | B |

| 50–59.99 | C |

| 40–49.99 | D |

| Below 40 | F |

11. Attendance System

The application considers **75%** as the attendance threshold.

```
```text
```

Attendance  $\geq$  75% → Good

Attendance  $<$  75% → Warning

```
```
```

This allows the generated report to provide an attendance status.

12. Performance Classification

The program classifies student performance as follows:

| Average | Classification |
|---------|----------------|
|---------|----------------|

| | |
|--------|-------|
| -----: | ----- |
|--------|-------|

| | |
|--------------|-----------|
| 75% or above | Excellent |
|--------------|-----------|

| | |
|-----------|------|
| 60–74.99% | Good |
|-----------|------|

| | |
|-----------|---------|
| 50–59.99% | Average |
|-----------|---------|

| | |
|-----------|-------------------|
| 40–49.99% | Needs Improvement |
|-----------|-------------------|

| | |
|-----------|------|
| Below 40% | Poor |
|-----------|------|

13. Input Validation

Input validation is a part of the application.

The program checks:

Duplicate Student

If the roll number already exists:

```text

Student already exists

...

### Invalid Marks

Marks outside 0–100 are rejected.

### Invalid Numerical Input

`try-except` is used to prevent the program from terminating when invalid numerical data is entered.

### Invalid Attendance

The program rejects:

- \* total classes

- \* Negative attended classes

- \* Attended classes greater, than classes

---

# 14. Sample Program Workflow

```text

Start

|

v

Display Menu

|

v

Select Operation

+---- Add Student

|

+---- Add Marks

|

+---- Add Attendance

|

+---- Generate Student Report

|

+---- Display Students

|

+---- Performance Analysis

|

+---- Exit

|

v

Continue?

|

Yes ----> Display Menu

|

No

|

v

End

...

15. Sample Output

A student report may appear in the following format:

```text

STUDENT REPORT

=====

Name: Manas

Roll No.: 101

Branch: CSE

Semester: 1

Python: 85.00

Mathematics: 78.00

Physics: 82.00

Total: 245.00

Average: 81.67%

Grade: A

Attendance: 88.00%

Attendance Status: Good

Best Subject: Python

Weakest Subject: Mathematics

Performance: Excellent

...

---

## # 16. Advantages

The project provides advantages:

- \* Simple and easy to use
- \* No external dependencies

- \* Command-line based and lightweight
- \* Reduces manual calculation
- \* Provides grade calculation
- \* Provides attendance analysis
- \* Identifies weakest subjects
- \* Provides basic class-level performance analysis
- \* Demonstrates use of Python programming concepts

---

## # 17. Limitations

The current implementation has some limitations:

1. Data is stored in memory.
2. Student data is lost after the program terminates.
3. There is no database.
4. There is no graphical user interface.

5. There is no authentication system.
6. Reports cannot currently be exported to files.
7. Student records cannot currently be. Deleted.

---

## # 18. Future Enhancements

The project can be improved by adding:

- \* File-based data storage
- \* SQLite/database integration
- \* GUI using Tkinter or another framework
- \* Student search functionality
- \* Edit and delete options
- \* PDF report generation
- \* CSV/Excel export
- \* Graphical performance charts
- \* Login and authentication

- \* Subject-wise class analysis

- \* Persistent student records

These additions could transform the basic application into a more complete student performance management system.

---

## # 19. Learning Outcomes

Through this project the following Python concepts were practiced:

- \* Variables

- \* Dictionaries

- \* Lists

- \* Functions

- \* Function parameters

- \* Return statements

- \* if-elif-

- \* for loops

- \* while loops

- \* try-except

- \* String formatting

- \* Dictionary methods

- \* sum()

- \* len()

- \* max()

- \* min()

- \* Lambda functions

- \* Input validation

- \* Basic data analysis

The project also demonstrates how programming concepts can be combined to solve an educational problem.

---

# 20.

The Student Performance Analyzer is a Python-based application for managing and analyzing student academic information.

The project successfully implements student registration, marks management, attendance calculation, grade calculation, report generation and class performance analysis.

It provides an application of fundamental Python programming concepts and serves as a foundation, for developing a more advanced student management system in the future.

---

## # 21. References

1. Python Programming Language Documentation
2. Python 3 Standard Library Documentation
3. Concepts of Python dictionaries, functions, loops, conditional statements and exception handling

---

## ## Project Information

**\*\*Project \*\*** Student Performance Analyzer

**\*\*Student:\*\*** Manas Pandey

**\*\*Registration ID:\*\* 26BCE10001**

**\*\*Branch:\*\* CSE – Educational Technology**

**\*\*Programming Language:\*\* Python**

**\*\*Project Type:\*\* Command-Line Application**